

LAB 01:

A group role named `cashier_role` was created and the database users `mike` and `jon` were assigned as members. The role was granted `SELECT` permission on `film` and `staff`, and `SELECT`, `INSERT` permissions on `payment`. This implements Role-Based Access Control and follows the Principle of Least Privilege by preventing cashiers from obtaining unnecessary administrative privileges.

Query

Query History

Scratch Pad

1

2

3

4

5

6

7

8

9

SELECT

r.rolname AS role_name,

m.rolname AS member_name

FROM pg_auth_members am

JOIN pg_roles r

ON am.roleid = r.oid

JOIN pg_roles m

ON am.member = m.oid

WHERE r.rolname = 'cashier_role';

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

	role_name	member_name
1	cashier_role	mike
2	cashier_role	jon

Total rows: 2

Query complete 00:00:00.404

CRLF

Ln 9, Col 34

Query

Query History

Scratch Pad

1

2

DELETE FROM film

WHERE film_id = 1;

Data Output

Messages

Notifications

ERROR: permission denied for table film

SQL state: 42501

Total rows: 2

Query complete 00:00:00.115

CRLF

Ln 2, Col 19

LAB 02:

dvdrental/postgres@PostgreSQL 18

Query Query History Scratch Pad

```
1 CREATE POLICY isolate_cashier_payments
2 ON payment
3 FOR SELECT
4 TO cashier_role
5 USING (
6     staff_id = (
7         SELECT staff_id
8         FROM staff
9         WHERE LOWER(username) = LOWER(current_user)
10    )
11 );
12 SELECT
13     staff_id,
14     username,
15     LOWER(username) AS username_lower
16 FROM staff;
17 SET ROLE mike;
18
19 SELECT current_user;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	staff_id smallint	visible_payments bigint
1	1	7292
2	2	7304

Total rows: 2 Query complete 00:00:00.155 CRLF Ln 43, Col 19

Query Query History Scratch Pad

```
21 SELECT
22     staff_id,
23     COUNT(*) AS visible_payments
24 FROM payment
25 GROUP BY staff_id;
26 SELECT
27     payment_id,
28     customer_id,
29     staff_id,
30     amount
31 FROM payment
32 LIMIT 10;
33 RESET ROLE;
34
35 SET ROLE jon;
36
37 SELECT current_user;
38
39 SELECT
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	staff_id smallint	visible_payments bigint
1	1	7292
2	2	7304

Total rows: 2 Query complete 00:00:00.155 CRLF Ln 43, Col 19

The screenshot shows a SQL IDE interface with a dark theme. The main window displays a SQL query with line numbers 25 to 43. The query is as follows:

```
25 GROUP BY staff_id;
26 SELECT
27     payment_id,
28     customer_id,
29     staff_id,
30     amount
31 FROM payment
32 LIMIT 10;
33 RESET ROLE;
34
35 SET ROLE jon;
36
37 SELECT current_user;
38
39 SELECT
40     staff_id,
41     COUNT(*) AS visible_payments
42 FROM payment
43 GROUP BY staff_id;
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'staff_id' (smallint) and 'visible_payments' (bigint). There are two rows of data:

	staff_id smallint	visible_payments bigint
1	1	7292
2	2	7304

The IDE interface includes a 'Query History' tab, a 'Scratch Pad' tab, and a 'Messages' tab. The status bar at the bottom indicates 'Total rows: 2', 'Query complete 00:00:00.155', and 'Ln 43, Col 19'.

After switching to the **mike** role, the RLS policy restricts the **payment** table so that Mike can only view rows associated with his own **staff_id**.

After switching to the **jon** role, only Jon's payment records are visible. This confirms that Row-Level Security successfully isolates transactions between cashiers.